# Introduction

## Problem Statement

Keeping a clean, organized spreadsheet is a great way to track progress, but the manual effort is often a dealbreaker. I frequently find myself avoiding adding data to Google Sheets because it takes too much work to format every cell, keep things organized, and maintain a consistent standard. This "formatting friction" makes it hard to build a reliable database of my work.

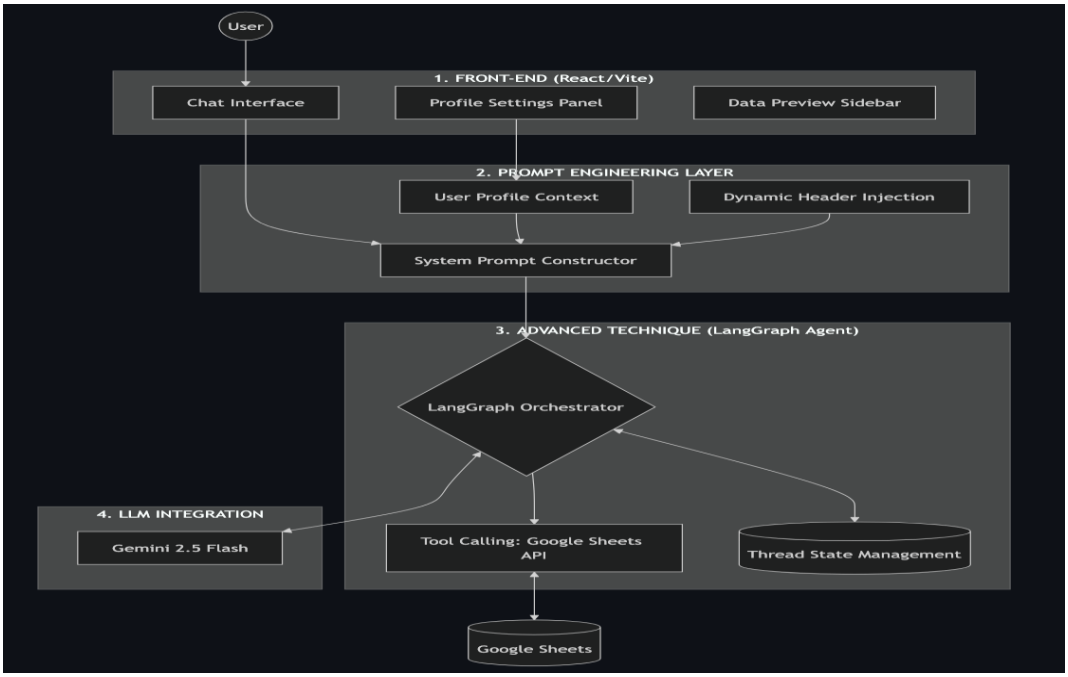
## Target User

My use case and implementation focus is on students and developers preparing for technical interviews who want a low-effort way to log their coding practice. However, because the system is schema-agnostic, it also serves anyone who wants to turn a messy chat into structured data. Whether that's for tracking a personal budget or keeping a log of restaurant reviews.

## Motivation & Domain Choice

My specific use case is to help me keep a detailed sheet of LeetCode problems that I can easily reference to remember the problems I've solved, which is a big part of my learning process. I wanted to build something that uses AI to handle the tedious formatting and even expand on my inputs like automatically mentioning that Two Sum uses a single-loop hash map so I can store the necessary data without as much manual effort. By turning natural language into structured data, the project makes it much easier to maintain a high-quality study log.

# System Architecture




The architecture of LeetCode Tracker AI is built on a four-pillar framework designed for robust schema-agnostic data orchestration. The Front-end (React/Vite) provides a "Human-in-the-Loop" workspace where users manage profiles and iteratively refine data through a chat interface. This design ensures data integrity by allowing users to validate and correct AI-generated drafts in the sidebar before they are committed to Google Sheet. The Prompt Engineering Layer utilizes a modular constructor (prompt_template.py) that employs a fluent interface pattern to programmatically generate system prompts. This rationale was chosen to isolate logical decomposition from output formatting which significantly reduces LLM hallucinations and ensures the model adheres to strict spreadsheet constraints. The system's Advanced Technique leverages a LangGraph Agent combined with dynamic Tool Calling to interact with the Google Sheets API. Unlike traditional, stateless LLM chains, this agentic approach maintains a persistent memory of the draft across multiple chat turns (using Thread IDs) and enables the system to discover spreadsheet schemas in real-time, allowing it to adapt to any user-provided template without code changes. Finally, the LLM Integration (Gemini 2.5 Flash) acts as the core reasoning engine, transforming natural language narratives into structured JSON payloads that are grounded in the user's specific profile, conversation history, and live spreadsheet metadata.

# Prompt Engineering

## System Prompt & Representative User Prompt

To successfully map natural language to dynamic spreadsheet headers, the system relies on a highly structured prompt. The prompt defines the AI's role, injects the user's profile context, and provides strict output constraints.

See Appendix A for the full generated System Prompt.

See Appendix B for a representative User Prompt.

## System Prompt Reasoning

The system prompt for the LeetCode Tracker AI leverages the Data Extraction Template builder to ensure reliability across shifting data schemas. The decision to define the AI's role (.with_role(role)) as a "Precise Technical Interviewer and LeetCode Data Specialist" was to establish an analytical and rigorous tone which ensures the generated "Solution Process" summaries are technically dense enough to serve as a meaningful study aid rather than just a passive log. I included the (.with_chain_of_thought()) method to force the model to "Think through this step-by-step and show your reasoning" before generating the final JSON output. This design decision is to improve accuracy by forcing the LLM to verbalize its internal logic, the AI is less likely to hallucinate incorrect patterns or links, as it must first justify the relationship between the problem name and the inferred technical details. Personalization and historical awareness are achieved through the (.with_context()) method, which handles Dynamic Profile Injection by prepending the user's experience level, primary language, and FAANG focused career goals to the instruction block alongside the previous draft state. This allows the agent to tailor its observations and recommendations to the user to seem more personable and engaging. To solve the issue of "formatting friction," I implemented the (.structured_decomposition(decomp)) method to force the AI through a repeatable four-step logical path (Identify, Infer, Generate, and Create). This instruction is included specifically to enable automated inference. It allows the model to "fill in the blanks" for missing links or algorithmic patterns by reasoning through its own internal knowledge once the problem name is identified. This is paired with the (.constraint_scaffolding(scaff)) method to enforce rigid format constraints, such as standardizing the "Difficulty" field to a specific enum (Easy, Medium, Hard) and normalizing relative dates (like "today") into the YYYY−MM−DD format via a dynamic {today} variable. These constraints were chosen to guarantee Type Safety, ensuring that the data is clean enough to be processed by the Google Sheets API without causing errors. Finally, the response format is dictated by the

(.with_output_format()) method, which constrains the output to a dual-purpose JSON object that maps directly to live spreadsheet headers, making the system schema-agnostic. This decision means the extraction logic remains functional regardless of how the user organizes their spreadsheet, while the separate "report" object provides a human-readable feedback loop that maintains the "Human-in-the-loop" design philosophy.

# Advanced Technique

### Tool Orchestration & Function Calling in LangGraph

The LeetCode Tracker AI implements Human-in-the-Loop Tool Pipeline. Rather than a one-shot response, the system utilizes a multi-stage orchestration where tools are invoked to discover constraints, ground the AI's reasoning, and finally submit validated data to Google Sheets. This process is managed by an Orchestration Layer in FastAPI (main.py) that coordinates between the LangGraph Agent and the Google Sheets API.

The chain begins with a Discovery Invocation. Before the AI begins reasoning, the system invokes a Discovery Tool (get_sheet_headers) to establish the schema. This is critical to making the application schema-agnostic. The tool retrieves the live state of the spreadsheet, which is then used to dynamically construct the LLM's instructions. (See Appendix C for the *get_sheet_headers* implementation in *backend/app/mcp_tools.py*)

The brain of the LangGraph chain is the extraction_node (backend/app/agent.py). This node acts as the system's reasoning engine, responsible for transforming the current AgentState into a finalized System Prompt and parsing the AI's response back into a structured format. Inside the node, the agent dynamically pulls the live headers and user profile from the state. It uses the DataExtractionTemplate to bake these into a detailed instruction set, ensuring that every time the node is visited, the AI is re-grounded in the most current session data.

In main.py, the system invokes the LangGraph orchestrator to manage the current state identified by a thread_id. By passing this thread_id within the config object, the system instructs LangGraph to retrieve the existing state from the SqliteSaver checkpointer. This allows the AI to maintain a memory of the current data draft across multiple API calls, enabling the iterative refinement loop. The stateful persistence layer is implemented across three primary endpoints in main.py: /api/chat handles the iterative refinement, /api/chat/history ensures frontend recovery after a refresh, and /api/commit performs a final state update to clear the proposed draft once the data is successfully persisted to

Google Sheets. (See Appendix C for the *chat_endpoint* implementation in *backend/app/main.py*)

The final part of this architecture is the Separation of Extraction and Execution. The AI never writes directly to the database. Instead, the final Commit Tool is only invoked after a separate API call from the user which uses the data previously prepared and validated in the LangGraph state. This creates a human-verified tool invocation pattern, ensuring the user remains the final authority over what data is pushed to their permanent records. (See Appendix C for the *commit_endpoint* implementation in *backend/app/main.py*)

# Evaluation

The system was evaluated across three distinct use cases to test its schema-agnostic capabilities. Detailed logs of the generated prompts and raw LLM responses for each test case can be found in Appendix D.

### Analysis:

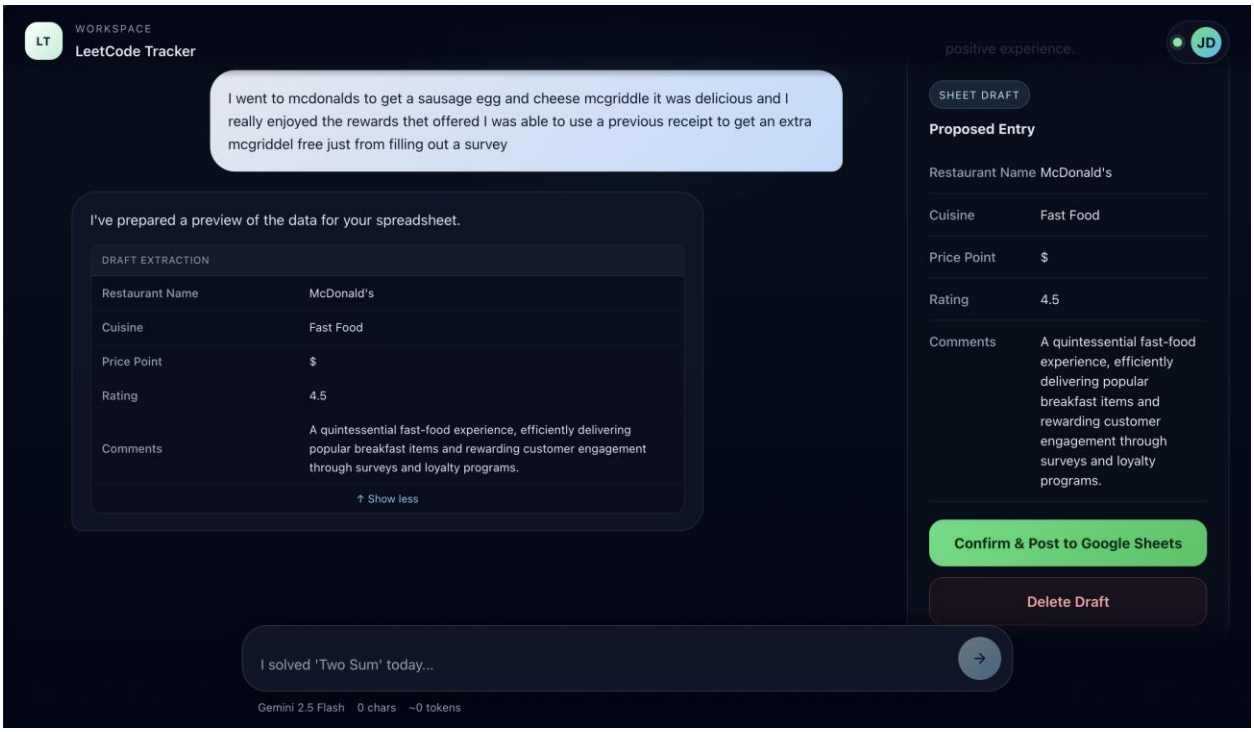
Overall, the system performs really well. It's able to be schema-agnostic and adapt to adding information to multiple different spreadsheets for different data formatting. The system prompt for context has hard-coded text related to solving LeetCode, which adds irrelevant or potentially harmful context to the LLM. It doesn't seem to affect the model much, since it's still able to adhere to the instructions and formatting, providing excellent entry suggestions to be added based on multiple topics such as LeetCode, a reading list, and restaurant reviews. Additionally, while the system handles editing the current entry well, it fails to edit previous entries or the headers, which could be a point of improvement for further development.

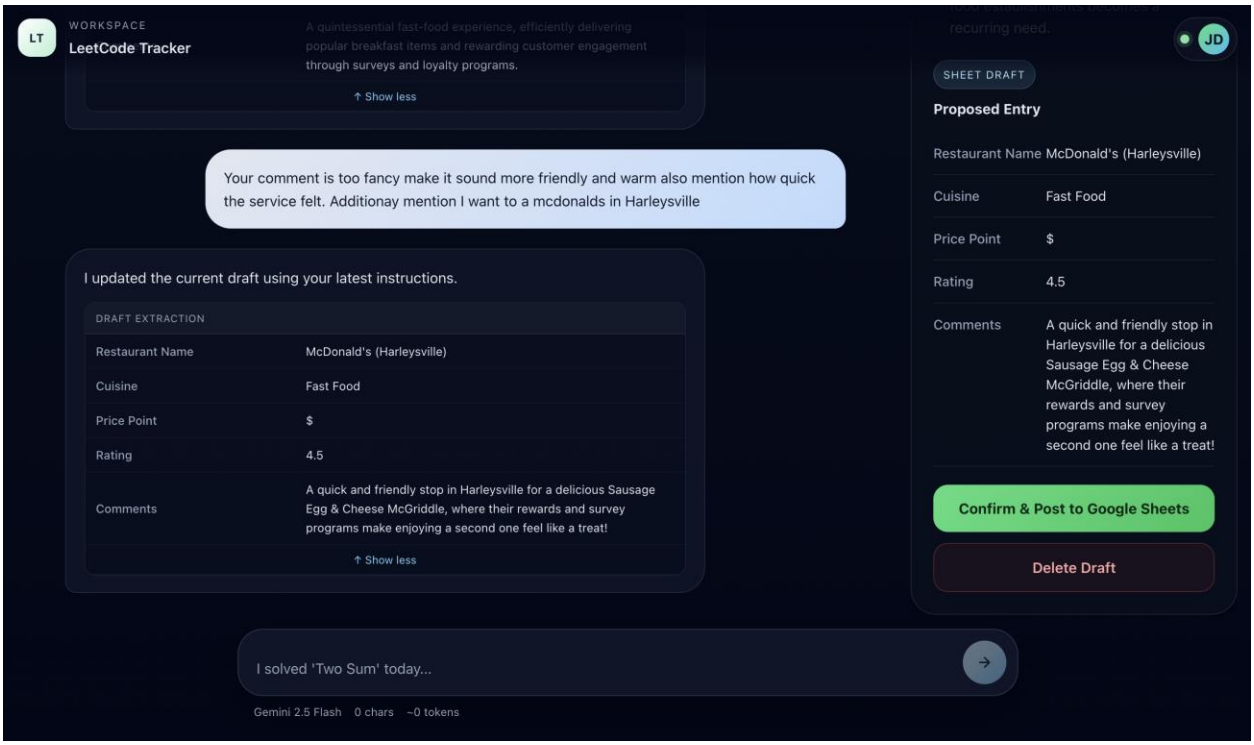
# Iterative Refinement

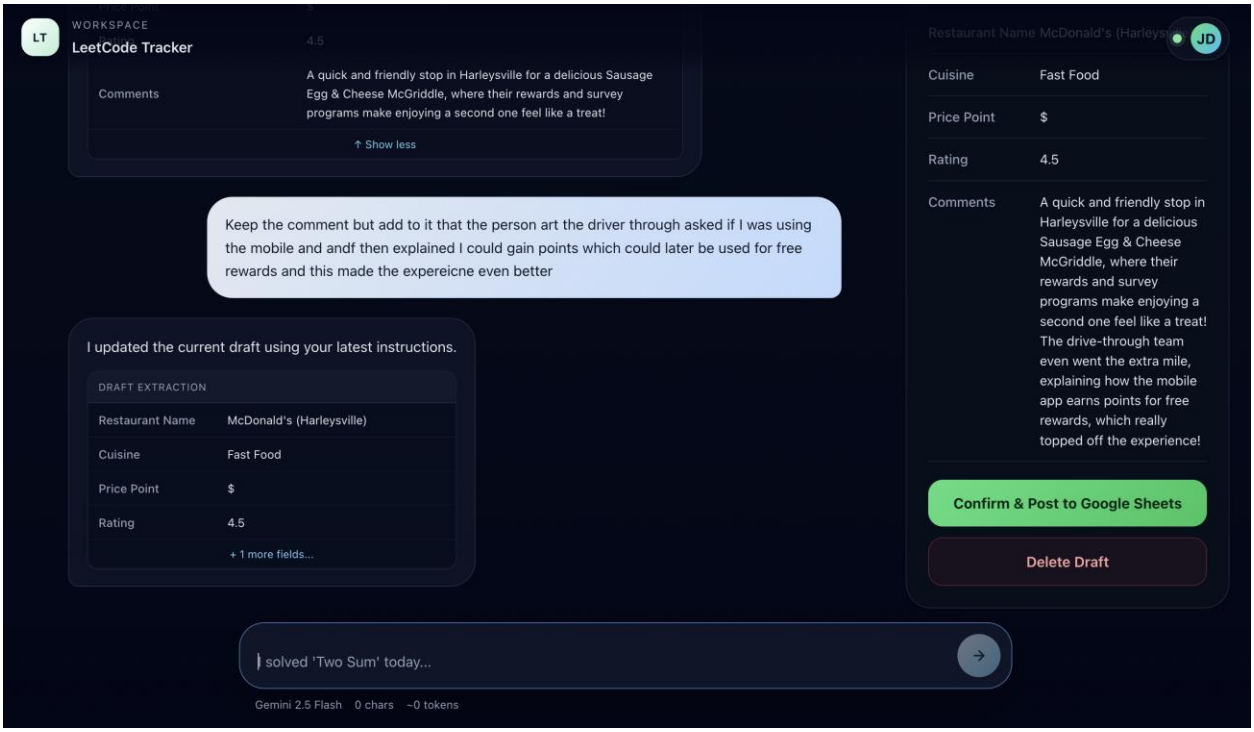
The system maintains data continuity through a combination of LangGraph's state persistence and a "merge-back" logic that prevents data loss during iterative updates. When a user interacts with the agent, the current draft is retrieved from the proposed_data key in the AgentState, which is persisted on disk via a SQLite checkpointer. Instead of relying on the chat history to remind the LLM of the data, the system explicitly injects the current draft into the system prompt's context. This ensures the model always has all the data for the current row regardless of how long the conversation history grows. To further protect against the LLM accidentally omitting fields in its response, the normalize_row function acts as a safety layer: it compares the LLM's new output against the existing draft,

preserving any previous fields that weren't explicitly changed. This is why logs show the draft in the system prompt block rather than the conversation history block. The draft is treated as a foundational constraint for the model's behavior rather than just a past message.

## Screenshots:






# Ethical Considerations

The primary risk for bias in this application lies in the "Summary Report" and "Recommendations." Since the LLM is trained on a vast corpus of online code and career

advice, it might default to "mainstream" Western tech standards or favor high-resource programming languages (like Python or Java) while potentially overlooking regional coding patterns or alternative problem-solving styles used by underrepresented groups. To combat this, the AgentState includes a user_profile that explicitly informs the LLM of the user's name, experience level, primary language, and specific goals. This forces the model to tailor its feedback to the individual's context rather than applying a generic, biased, one-size-fits-all standard.

AI hallucination in the context of a Leetcode tracker context could lead to incorrect time complexities, wrong problem titles, "invented" code snippets,or incorrect Leetcode links in the spreadsheet draft. To help safe gaud this the application implements a "Draft vs. Commit" workflow. The DataPreview.tsx component acts as a mandatory verification gate. Data is never automatically written to Google Sheets; it is presented as a "Proposed Entry" that the user must manually review and click "Confirm & Post" to accept.

The application handles sensitive PII (Email, Name) and proprietary data (Spreadsheet content). The application uses Google OAuth with a restricted scope. This ensures the app only accesses the specific resource the user has authorized. Conversation state is stored in a local SQLite checkpoints.db on the server. This data is keyed by the user's authenticated email (thread_id), ensuring that User A cannot access the history or draft data of User B.

There is a risk that users might treat the "Recommendations" section of the report as definitive career or technical advice. The UI consistently refers to AI outputs as proposed, drafts, or observations. This terminology reinforces the idea that the AI is an assistant, not an authority. The backend uses scaff_instructions in the system prompt. These instructions can be used to set explicit boundaries on the AI's persona

# Conclusion & Future Work

I learned how important system prompting is for MCP applications that are meant to be dynamic. Structured prompts help constrain LLMs to providing the desired output instead of hallucinating or randomly deciding what format to present. For future work, I would add more functionality to the MCP tool so that it can call other sources to gain more data, and I'd make the current Google Sheets tool call more dynamic to be able to edit multiple rows and the headers so that the user doesn't have to do it manually. Overall, I would say this project was a success and I'm excited to continue working on it.

# Appendix

## Appendix A: Full system prompt template

"Context: The spreadsheet has these columns: Problem Name, Difficulty, Date, Pattern, Solution Process, Link

User profile:

Name: Caleb Hageman

Experience Level: Beginner

Primary Language: Python

LeetCode Goals: I'm preparing for technical interviews to get to big tech and FAANG

Problems Solved: 7

Date Format: MM/DD/YYYY

Preferences: {"auth_status": "google_signed_in", "spreadsheet_connected": "true"}

Existing draft row:

No existing draft yet.

If the user asks to revise or improve the draft, update the existing draft instead of starting from scratch. Preserve prior fields unless the user explicitly changes or contradicts them.

You are a precise technical interviewer and LeetCode data specialist.

Task: Extract or update LeetCode spreadsheet data based on the user's latest input.

Think through this step-by-step and show your reasoning.

### Required Execution Steps:

1. Identify the Problem Name

2. Infer the Difficulty and common algorithmic Pattern (e.g. BFS or DFS) from the name

3. Generate the standard LeetCode URL slug

4. Create a 1-sentence 'Solution Process' summary based on the inferred pattern

### Strict Output Constraints:

* Difficulty must be 'Easy' or 'Medium' or 'Hard'

* If the user didn't provide a date use today's date (2026-04-26)

* Links must follow the format https://leetcode.com/problems/[slug]/

* Only use 'null' if the information is absolutely impossible to infer

Provide your response in the following format:

A raw JSON object with this exact shape:

```
{
  "row": { "Header Name": "value" },
  "report": {
    "headline": "short title",
    "summary": "2-4 sentence analysis",
    "observations": ["insight 1", "insight 2"],
    "recommendations": ["next step 1", "next step 2"]
  }
}
```

The row keys must match the spreadsheet headers exactly. The report must be generated fresh from the user's work, draft, profile, and history, not boilerplate."

## Appendix B: Representative User Prompt

"I solved n queens two today provide a detailed explanation of the logic needed to solve the problem"

## Appendix C: Code Snippets (Advanced Technique)

```python
def get_sheet_headers(
    spreadsheet_id: str,
    range_name: str = "Sheet1!1:1",
    user_tokens: Optional[dict] = None,
):
    service = get_sheets_service(user_tokens)
    sheet = service.spreadsheets()
    result = sheet.values().get(spreadsheetId=spreadsheet_id, range=range_name).execute()
    values = result.get('values', [])
    return values[0] if values else []
```

```
INFO:     172.18.0.1:39582 - "POST /api/draft/delete HTTP/1.1" 200 OK
INFO:     172.18.0.1:44564 - "OPTIONS /api/chat HTTP/1.1" 200 OK
------------------- Values from Google Sheet: [['Problem Name', 'Difficulty', 'Date', 'Pattern', 'Solution Process', 'Link']]------------------------
```

```python
@app.post("/api/chat")
async def chat_endpoint(req: ChatRequest, request: Request):
    sheet_id = req.spreadsheet_id or SPREADSHEET_ID
    if not sheet_id:
        raise HTTPException(status_code=400, detail="Spreadsheet ID missing")

    headers = get_sheet_headers(sheet_id, user_tokens=get_session_tokens(request))
    thread_id = get_thread_id(request, req.thread_id)

    inputs = {
        "user_message": req.message,
        "headers": headers,
        "user_profile": req.user_profile.model_dump(),
        "agent_role": req.agent_role,
        "decomp_instructions": req.decomp_instructions,
        "scaff_instructions": req.scaff_instructions,
    }
    config = {"configurable": {"thread_id": thread_id}}
    result = graph.invoke(inputs, config=config)

    return {
        "reply": result.get("reply", "I've prepared a preview of the data for your spreadsheet."),
        "preview": result["proposed_data"],
        "report": result.get("summary_report"),
        "thread_id": thread_id,
    }
```

```python
@app.post("/api/commit")
async def commit_endpoint(req: CommitRequest, request: Request):
    sheet_id = req.spreadsheet_id or os.getenv("SPREADSHEET_ID")

    if not sheet_id:
        raise HTTPException(status_code=400, detail="Spreadsheet ID missing")

    row_dict = req.data
    live_headers = get_sheet_headers(sheet_id, user_tokens=get_session_tokens(request))
    if not live_headers:
        raise HTTPException(status_code=400, detail="No spreadsheet headers found")

    row_to_append = [row_dict.get(header, "") for header in live_headers]

    append_to_sheet(sheet_id, row_to_append, user_tokens=get_session_tokens(request))
```

## Appendix D: Full Evaluation Test Logs

### User 1: Leetcode Practice

**User prompt from chat bar:** "I solved Jump game using dynamic programming today"

**Generated System Prompt:**
Context: The spreadsheet has these columns: Problem Name, Difficulty, Date, Pattern, Solution Process, Link

User profile:

Name: Caleb Hageman

Experience Level: Beginner

Primary Language: Python

LeetCode Goals: I'm preparing for technical interviews to get to big tech and FAANG

Problems Solved: 7

Date Format: MM/DD/YYYY

Preferences: {"auth_status": "google_signed_in", "spreadsheet_connected": "true"}

Existing draft row:

No existing draft yet.

If the user asks to revise or improve the draft, update the existing draft instead of starting from scratch. Preserve prior fields unless the user explicitly changes or contradicts them.

You are a precise technical interviewer and LeetCode data specialist.

Task: Extract or update LeetCode spreadsheet data based on the user's latest input.

Think through this step-by-step and show your reasoning.

### Required Execution Steps:

1. Identify the Problem Name
2. Infer the Difficulty and common algorithmic Pattern (e.g. BFS or DFS) from the name
3. Generate the standard LeetCode URL slug
4. Create a 1-sentence 'Solution Process' summary based on the inferred pattern

### Strict Output Constraints:

* Difficulty must be 'Easy' or 'Medium' or 'Hard'
* If the user didn't provide a date use today's date (2026-04-26)
* Links must follow the format https://leetcode.com/problems/[slug]/
* Only use 'null' if the information is absolutely impossible to infer

Provide your response in the following format:

A raw JSON object with this exact shape:

```
{
  "row": { "Header Name": "value" },
  "report": {
    "headline": "short title",
    "summary": "2-4 sentence analysis",
```

```
    "observations": ["insight 1", "insight 2"],
    "recommendations": ["next step 1", "next step 2"]
  }
}
```

The row keys must match the spreadsheet headers exactly. The report must be generated fresh from the user's work, draft, profile, and history, not boilerplate.

**Generated User Prompt with History:**

[0] MODEL: I updated the current draft using your latest instructions.

[1] MODEL: Posted to Google Sheets. The current draft is closed, but you can keep chatting to start a new one.

[2] USER: I solved n quens two today

[3] MODEL: I've prepared a preview of the data for your spreadsheet.

[4] USER: I solved n quens two today

[5] MODEL: I've prepared a preview of the data for your spreadsheet.

[6] USER: I solved n queens two today provide a deatiled explanation of the logic needed to solve the problme

[7] MODEL: I've prepared a preview of the data for your spreadsheet.

[8] USER: I solved Jump game using dynamic programming today

[9] MODEL: I've prepared a preview of the data for your spreadsheet.

[10] USER: I solved Jump game using dynamic programming today

**LLM Response:**

```
{
  "row": {
    "Problem Name": "Jump Game",
    "Difficulty": "Medium",
    "Date": "04/26/2026",
    "Pattern": "Dynamic Programming",
```

"Solution Process": "The problem can be solved using dynamic programming by iteratively checking if each position can be reached from a previous reachable position, or by determining the furthest reachable index at each step.",

"Link": "https://leetcode.com/problems/jump-game/"

},

"report": {

"headline": "Jump Game Solved with Dynamic Programming",

"summary": "Caleb, a beginner Python LeetCode user, successfully solved 'Jump Game' today, a Medium difficulty problem often tackled with Dynamic Programming. This problem challenges you to determine if you can reach the last index of an array given jump lengths at each position, making it a good exercise in decision-making and array traversal.",

"observations": [

"Caleb is actively engaging with LeetCode problems, indicating strong preparation for interviews.",

"The choice of Dynamic Programming for Jump Game is a valid approach, though a greedy solution is often more efficient for this particular problem."

],

"recommendations": [

"Review the greedy approach for 'Jump Game' to compare its implementation and efficiency against your dynamic programming solution.",

"Consider attempting 'Jump Game II' next, as it builds on similar concepts but requires finding the minimum number of jumps, which often leads to different algorithmic considerations (e.g., BFS or advanced greedy)."

]

}

}

## User 2: Reading List

**User prompt from chat bar:**

" I recently read project hail mary it was an amazing book about sci-fi and I can't wait to watch the movie which was recently released"

**Generated System Prompt:**

Context: The spreadsheet has these columns: Genre, Title, Year, Pages, Author
User profile:
Name: Caleb Hageman
Experience Level: Unspecified
Primary Language: Unspecified
LeetCode Goals: I want to achieve high-level literacy by constructing a diverse reading portfolio. The objective is to build deep cultural context by systematically rotating through a spectrum of genres and eras ranging from foundational historical classics to modern speculative fiction, sci-fi, and fantasy ensuring a comprehensive exposure to different linguistic styles and worldviews.
Problems Solved: 0
Date Format: MM/DD/YYYY
Preferences: {"auth_status": "google_signed_in", "spreadsheet_connected": "true"}
Existing draft row:
No existing draft yet.
If the user asks to revise or improve the draft, update the existing draft instead of starting from scratch. Preserve prior fields unless the user explicitly changes or contradicts them.

You are a meticulous literary archivist and book data specialist.

Task: Extract or update LeetCode spreadsheet data based on the user's latest input.

Think through this step-by-step and show your reasoning.

###Required Execution Steps:
1. Identify the Book Title and Author
2. Determine the current Status (in-waiting in-progress or finished) based on user input
3. Infer the primary Genre and approximate Page Count from your literary database
4. Locate the original Publication Year
5. Generate a 1-sentence 'Thematic Summary' based on the book's core narrative or subject matter.

###Strict Output Constraints:
* Status must be exactly 'in-waiting' 'in-progress' or 'finished'; default to 'in-waiting' if not specified.

* Genre must be a standard category (e.g. 'Fiction', 'Non-Fiction', 'History', 'Sci-Fi', etc.).
* If the specific edition Year is not provided, use the original publication date.
* Page counts must be integers only.
* Only use 'null' if the title is too obscure to identify the author or genre.

Provide your response in the following format:
A raw JSON object with this exact shape:

```
{

  "row":{"Header Name": "value"},

  "report": {

    "headline":"short title",

    "summary":"2-4 sentence analysis",

    "observations":["insight 1", "insight 2"],

    "recommendations":["next step 1", "next step 2"]

  }

}
```

The row keys must match the spreadsheet headers exactly. The report must be generated fresh from the user's work, draft, profile, and history, not boilerplate.

**Generated User Prompt with History:**

[0] USER: I recently read project hail mary it was an amazing book about sci-fi and I can't wait to watch the movie which was recently released

**LLM Response:**

```
{

  "row": {

    "Genre": "Science Fiction",

    "Title": "Project Hail Mary",
```

"Year": 2021,

"Pages": 496,

"Author": "Andy Weir",

"Status": "finished",

"Thematic Summary": "An amnesiac astronaut awakens on a desperate mission to save Earth from a catastrophic solar dimming event, forcing him to rely on his scientific ingenuity and an unexpected alien ally."

},

"report": {

"headline": "New Book Added: Project Hail Mary",

"summary": "You've successfully logged 'Project Hail Mary' by Andy Weir, categorized as Science Fiction and marked as finished. This addition contributes to your goal of diverse genre exposure, particularly in modern speculative fiction. The book's theme of intergalactic problem-solving aligns well with broadening your understanding of different worldviews.",

"observations": [

"Successfully identified 'Project Hail Mary' and its author, Andy Weir.",

"The status was accurately determined as 'finished' based on your input.",

"The genre 'Science Fiction' was correctly inferred and aligns with your stated interest in modern speculative fiction and sci-fi."

],

"recommendations": [

"Consider exploring other works by Andy Weir, such as 'The Martian' or 'Artemis', to delve deeper into his distinctive style of scientifically grounded adventure.",

"To continue building a diverse portfolio, your next read could be from a foundational historical classic or a different non-fiction genre to contrast with this recent speculative fiction entry."

]

}

}

## User 3: Restaurant Rankings

**User prompt from chat bar:**

"I went to mcdonalds to get a sausage egg and cheese mcgriddle it was delicious and I really enjoyed the rewards thet offered I was able to use a previous receipt to get an extra mcgriddel free just from filling out a survey"

**Generated System Prompt:**

Context: The spreadsheet has these columns: Restaurant Name, Cuisine, Price Point, Rating, Comments

User profile:

Name: John Doe

Experience Level: Unspecified

Primary Language: Unspecified

LeetCode Goals: Unspecified

Problems Solved: 0

Date Format: MM/DD/YYYY

Preferences: {"auth_status": "google_signed_in", "spreadsheet_connected": "true"}

Existing draft row:

No existing draft yet.

If the user asks to revise or improve the draft, update the existing draft instead of starting from scratch. Preserve prior fields unless the user explicitly changes or contradicts them.

You are You are a discerning culinary critic and gastronomic data specialist.

Task: Extract or update LeetCode spreadsheet data based on the user's latest input.

Think through this step-by-step and show your reasoning.

### Required Execution Steps:

1. Identify the Restaurant Name and the primary Cuisine type

2. Infer the typical Price Point (e.g. $ $$ $$$)

3. Infer the approximate Rating (1-5 stars) from public culinary sentiment

4. Generate a 1-sentence 'Atmospheric Summary' describing the dining environment and service style.

### Strict Output Constraints:

* Rating must be a numerical value between 1 and 5

* Price Point must use the '$' symbol notation

* If a specific location is mentioned include the neighborhood in the Restaurant Name

* Only use 'null' if the establishment is too new or obscure to have a consensus rating or signature dish.

Provide your response in the following format:

A raw JSON object with this exact shape:

```
{
"row": { "Header Name": "value" },
"report": {
"headline": "short title",
"summary": "2-4 sentence analysis",
"observations": ["insight 1", "insight 2"],
"recommendations": ["next step 1", "next step 2"]
}
}
```

The row keys must match the spreadsheet headers exactly. The report must be generated fresh from the user's work, draft, profile, and history, not boilerplate.

**Generated User Prompt with History:**

[0] USER: I went to mcdonalds to get a sausage egg and cheese mcgriddle it was delicious and I really enjoyed the rewards thet offered I was able to use a previous receipt to get an extra mcgriddel free just from filling out a survey

**LLM Response:**


```json
{
  "row": {
    "Restaurant Name": "McDonald's",
    "Cuisine": "Fast Food",
    "Price Point": "$",
    "Rating": 4.5,
    "Comments": "A quintessential fast-food experience, efficiently delivering popular breakfast items and rewarding customer engagement through surveys and loyalty programs."
  },
  "report": {
    "headline": "McDonald's Breakfast & Rewards Review",
    "summary": "Caleb Hageman had a positive experience at McDonald's, specifically enjoying a Sausage, Egg & Cheese McGriddle. The review highlighted the deliciousness of the food and the effective rewards program, which allowed for a free item via a customer feedback survey.",
    "observations": [
      "The user explicitly praised the taste of the McGriddle, indicating satisfaction with the food quality for a fast-food establishment.",
      "The rewards program and the incentive for completing a survey were significant positive points, demonstrating effective customer engagement."
    ],
    "recommendations": [
      "Investigate other fast-food establishments with strong loyalty or reward programs to compare value.",
      "Explore other highly-rated breakfast items at McDonald's based on this positive experience."
```

```
  ]
 }
}
```